GS2.401 SPATIAL INFORMATICS · MID-SUBMISSION REPORT

Large-Scale Terrain Data Analysis 2

Radar visibility, flight-route visibility and radar-avoiding corridors over the Garhwal Himalaya

Track 1 · Radar visibility	Person 1
Track 2 · Flight-route visibility	Person 2
Track 3 · Visibility corridor	Person 3

October 2026

Copernicus GLO-30 DEM of the study area, 120 x 120 km, rendered in QGIS

Contents

Executive Summary 3

1. Introduction 4

1.1 Objectives 4
1.2 The three tracks 4

2. Data and Shared Methodology 5

2.1 Study area and elevation data 5
2.2 Visibility model 5
2.3 Benchmark protocol 6

3. Track 1: Radar Visibility Assessment 7

3.1 Design and implementation 7
3.2 Validation 7
3.3 Scenario: six ridge-top radar sites 8
3.4 First benchmark: cost against DEM size (E1) 8

4. Track 2: Flight-Route Visibility 10

4.1 Design and implementation 10
4.2 Scenario: valley flying against cruise 10
4.3 First benchmark: cost against DEM size (F1) 11

5. Track 3: Optimal Visibility Corridor 12

5.1 Objective 12
5.2 Solvers 12
5.3 Scenario: Devprayag to Joshimath 12
5.4 First benchmark: search time against graph size (C1) 13

6. Progress Against the Plan 15

7. Future Work 16

7.1 Track 1: radar visibility 16
7.2 Track 2: flight-route visibility 16
7.3 Track 3: optimal visibility corridor 16
7.4 Across the tracks 16
7.5 Timeline to the final submission 17

8. Conclusions 18

References 18

Appendix A. Results Beyond the Mid-Submission Scope 19

A.1 Track 1: parameters and optimisations 19
A.2 Track 2: parameters and optimisations 20
A.3 Track 3: parameters and optimisations 22
A.4 Visualisation performance 23
A.5 Comparison across the tracks 24

Appendix B. Track 3: Interactive Planning 25

B.1 Compiled search 25
B.2 What to precompute: a ladder of levels 25
B.3 Building the field on 12 cores 26
B.4 Edits: moving the start, a radar, the goal 26
B.5 When the precompute stops being valid: changing alpha 27
B.6 Panning and zooming 28
B.7 Latency budget 28
B.8 From exposure to detection probability 29
B.9 A precompute that survives edits: tile overlay 31

Executive Summary

We are building three terrain-visibility applications on one shared set of elevation models of the Garhwal Himalaya and measuring how their cost grows with terrain size and problem settings. **Track 1** computes what a stationary radar can see, **Track 2** what an aircraft sees along a 166 km valley route, and **Track 3** plans a route that keeps the aircraft hidden from the Track 1 radars. All three run end to end, pass correctness checks, export GIS-ready outputs and have been benchmarked from 0.25 million to 16 million terrain cells.

Key results so far

99.8%
agreement of the production viewshed (GDAL) with the exact line-of-sight answer

0.10 s
for a 16 M-cell viewshed with GDAL, against about 68 min for the exact method

16.7%
of the 120 x 120 km area is visible to six ridge-top radars: valleys hide most terrain

25.5%
of the route corridor seen when flying low in the valley, against 78.6% at a 5,500 m cruise

-93.1%
radar exposure of the optimised route, for 30.2% more distance

8x
faster route search in C than in Python on 16 M nodes, at 9x the memory

Scope of this mid-submission

Sections 1-6 cover the agreed first half of the project for every track: shared data and protocol, design, implementation, validation, maps and the first scaling benchmark. Section 7 sets out the work planned for the final submission. Appendix A reports experiments for the second half that have already been run (parameter studies, optimisations, rendering performance); they are included for completeness and will be extended for the final. Appendix B reports the Track 3 work done since: interactive planning (what to precompute and what to compute on the fly) and extensions of the corridor model.

1. Introduction

Line-of-sight over terrain decides what a radar can cover, what an aircraft can observe and where it can fly unseen. The computation itself is simple, but its cost grows quickly with the size and resolution of the terrain and with the number of observers. The project brief asks for three such applications, built and evaluated for **computational complexity, memory efficiency, visualisation performance, accuracy and scalability**, and for an explanation of *how and why* their demands grow and which implementation choices help.

1.1 Objectives

- Build three working terrain-visibility applications on shared digital elevation models (DEMs).
- Produce GIS outputs and maps of visible and blocked regions, routes and corridors.
- Benchmark runtime, peak memory, scalability, complexity, rendering performance and accuracy.
- Compare baselines with optimised implementations and explain the trade-offs.

1.2 The three tracks

Table 1. Division of work. Each track is separately executable and owned end to end by one member.

Track	Owner	Question	Main output
1 Radar visibility	Person 1	Which terrain cells can a stationary radar see?	Coverage rasters and maps
2 Flight-route visibility	Person 2	How does visibility change along a moving flight route?	Cumulative coverage, blind spots
3 Visibility corridor	Person 3	Which feasible route best balances radar exposure and distance?	Optimised routes and trade-offs

The tracks share data and conventions, and are linked deliberately: Track 3 avoids the radar sites of Track 1 and plans between the endpoints of the Track 2 route, so the three results can be read together.

2. Data and Shared Methodology

2.1 Study area and elevation data

We use the Copernicus GLO-30 digital surface model (30 m), reprojected with bilinear resampling to WGS 84 / UTM zone 44N (EPSG:32644). Four square DEMs share one centre (Rudraprayag) and the same 30 m cells, so a larger DEM covers more ground rather than finer detail, which isolates the effect of problem size. The area runs from the Ganga foothills to the Nanda Devi massif.

Table 2. The four benchmark DEMs (no no-data cells).

DEM	Cells	Extent	Min (m)	Max (m)	Relief (m)	Std (m)
500 x 500	250,000	15 x 15 km	594	2,856	2,263	448
1000 x 1000	1,000,000	30 x 30 km	547	3,022	2,475	479
2000 x 2000	4,000,000	60 x 60 km	485	4,480	3,995	556
4000 x 4000	16,000,000	120 x 120 km	315	7,074	6,759	1,122

Figure 1. The 4000 x 4000 DEM (120 x 120 km): hillshade over elevation, rendered in QGIS.

> **Note**
> Larger DEMs also reach higher, more rugged terrain (relief grows from 2.3 km to 6.8 km), so size and terrain complexity change together. Section 7 plans a second, flatter region to separate the two effects.

2.2 Visibility model

A cell is **visible** if the point *target height* above it can be joined to the observer's eye, *observer height* above its cell, by a straight line that no terrain rises above. Earth curvature with standard refraction lowers distant

terrain by $(1 - k)\cdot d^2/2R$ with k = 1/7. The same model is used by every algorithm and track:

Table 3. Shared visibility parameters (common/config.py).

Parameter	Value	Used by
Radar mast height	30 m above ground	Tracks 1, 3
Terrain target height	0 m (the ground surface)	Tracks 1, 2
Aircraft target height	150 m above ground	Track 3 exposure
Radar range	40 km (scenarios)	Tracks 1, 3
Flight clearance / sensor radius	300 m above ground / 20 km	Track 2
Output coding	1 visible, 0 blocked, 255 out of range	all rasters

2.3 Benchmark protocol

Each measured run is a fresh Python process (*common/bench.py*), so memory from one run cannot leak into the next. Runs are repeated five times (three or one for the slowest configurations); we report the median and keep every run in the CSV files. **Compute time** covers the algorithm only, **runtime** also includes reading the DEM, and **peak memory** is the peak working set of the process (about 60 MB of which is Python, NumPy and GDAL themselves). Empirical complexity is the slope of log(time) against log(size), compared with the theoretical order.

Table 4. Benchmark machine and software.

Component	Version / specification
Processor	AMD Ryzen 9 7900X, 12 cores / 24 threads
Memory	31.1 GB
Operating system	Windows 11
GIS	QGIS 3.44.12, GDAL 3.13.1
Python stack	Python 3.12.13, NumPy 2.4.6, SciPy 1.17.1

3. Track 1: Radar Visibility Assessment

PERSON 1

3.1 Design and implementation

Three viewshed algorithms sit behind one interface (*radar/viewshed.py*) with the same physical model and output, so every benchmark can swap them freely.

Table 5. The three viewshed algorithms (N = cells in range).

Algorithm	Idea	Complexity	Role
Naive line of sight	For every cell, sample the terrain profile to the observer every half cell (bilinear) and test whether anything rises above the sight line	O(N·√N)	exact reference
R2 ray casting	One ray to every cell on the boundary square; each ray keeps a running horizon (Franklin and Ray). Our own vectorised NumPy code	O(N)	own fast method
GDAL viewshed	C++ reference-plane algorithm (Wang et al.)	O(N)	production engine

3.2 Validation

Table 6. Correctness on synthetic terrains with known answers, and against the exact reference on real terrain.

Test	Naive	R2	GDAL
Flat plane: all cells visible	100.0%	100.0%	100.0%
V-shaped valley: all cells visible	100.0%	100.0%	100.0%
200 m wall: everything behind it hidden	100.0%	100.0%	100.0%
Real DEM 500, agreement with exact	reference	99.7%	99.8%
Real DEM 500, Cohen's kappa	reference	0.957	0.965
Real DEM 1000, 10 km range, agreement	reference	99.8%	99.8%

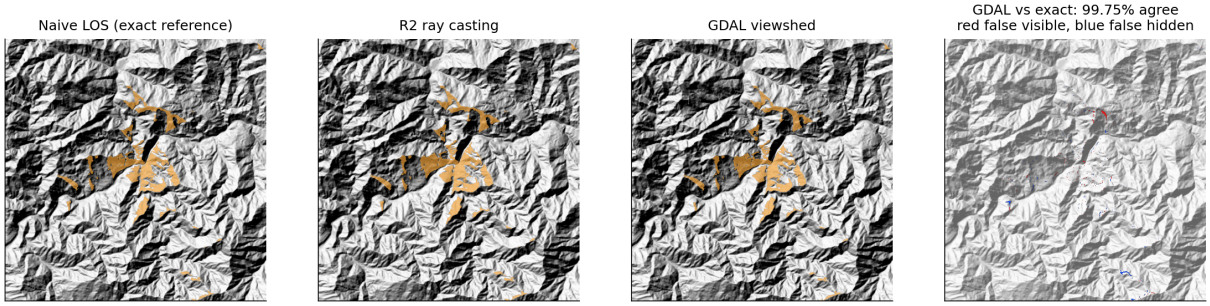

Figure 2. The same radar computed three ways (DEM 500). Right: cells where GDAL differs from the exact answer; they lie on grazing ridge edges.

Bug found by validation

An early R2 version scored 84.7% on the valley: it compared the cell centre against a horizon built from interpolated ray points. Testing the ray point itself fixed it and raised agreement on real terrain to 99.7%.

3.3 Scenario: six ridge-top radar sites

Sites are chosen by a fixed rule (*common/sites.py*): the highest point within 3 km, between 1,500 and 3,200 m, at least 22 km apart. Each radar has a 40 km range.

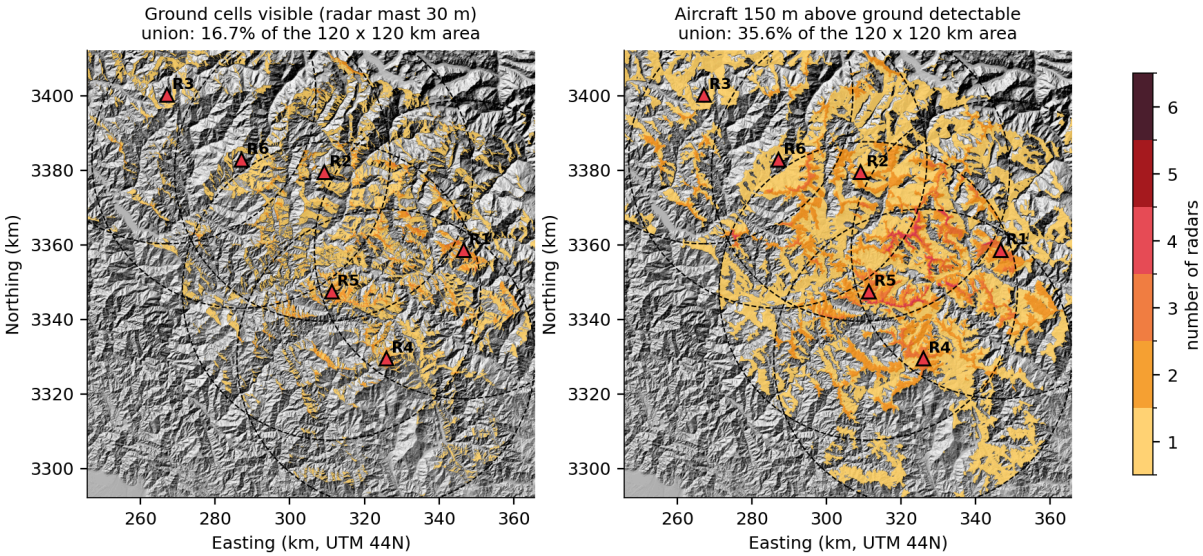

Figure 3. Number of radars that see each ground cell (left) and each aircraft position 150 m above ground (right). Dashed circles: 40 km range.

> **Finding**
>
> Together the six radars see only **16.7%** of the ground but would detect a low aircraft over **35.6%** of the area. A single radar sees 3.8% to 14.4% of the ground in its range: deep valleys hide most terrain even from ridge tops.

3.4 First benchmark: cost against DEM size (E1)

Table 7. Radar at the DEM centre with unlimited range (the whole DEM is processed); median compute time.

DEM cells	Naive (s)	R2 (s)	GDAL (s)	GDAL peak memory
250,000	6.62	0.027	0.004	68 MB
1,000,000	56.37	0.088	0.010	72 MB
4,000,000	not run	0.326	0.031	103 MB
16,000,000	~4,087 (projected)	1.585	0.100	242 MB

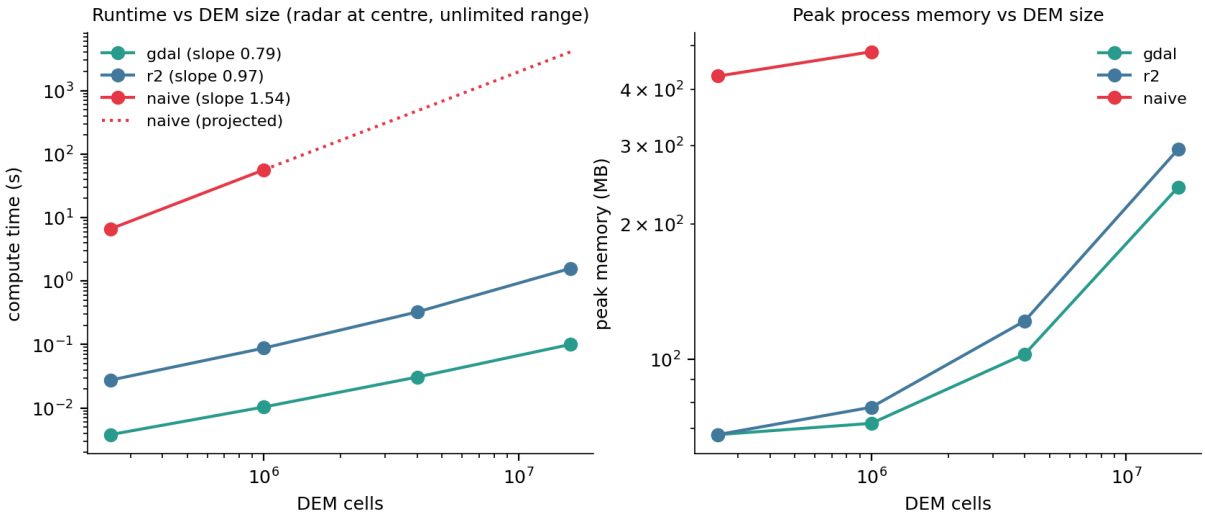

Figure 4. Log-log compute time and peak memory against DEM size; the slope is the empirical complexity exponent.

Finding

Measured exponents match the theory: naive $N^{1.54}$ (theory $N^{1.5}$), R2 $N^{0.97}$ and GDAL $N^{0.79}$ (theory N; fixed overheads flatten the smallest sizes). The exact method would need about 68 minutes for the 16 M-cell DEM against 0.10 s for GDAL, while agreeing to within 0.3%. Memory follows the DEM and output arrays, about 5 bytes per cell.

4. Track 2: Flight-Route Visibility

PERSON 2

4.1 Design and implementation

Table 8. Flight-visibility pipeline (flight/route.py, flight/visibility.py).

Step	What we do
Route	A 166 km route up the Alaknanda valley (Devprayag, Srinagar, Rudraprayag, Karnaprayag, Chamoli, Joshimath), found as the least-elevation path between the towns and simplified to 300 m
Altitude	Terrain following 300 m above a smoothed ground (running maximum over 3 km), or a constant cruise altitude; computed once at 100 m spacing so every sampling density flies the same path
Sampling	A position every 1,000 m (167 positions), a viewshed within a 20 km sensor radius at each (Track 1's GDAL engine)
Aggregation	Times seen per cell, first position that sees it, visible and newly seen area per position, and blind spots: cells within the radius of the route that are never seen

4.2 Scenario: valley flying against cruise

Table 9. Coverage of the 6,024 km² corridor within 20 km of the route.

Flight profile	Area seen	Corridor seen	Blind in corridor	Mean visible per position
Terrain following, 300 m AGL	1,537 km²	25.5%	4,487 km²	51 km²
Cruise, 5500 m MSL	4,732 km²	78.6%	1,292 km²	698 km²

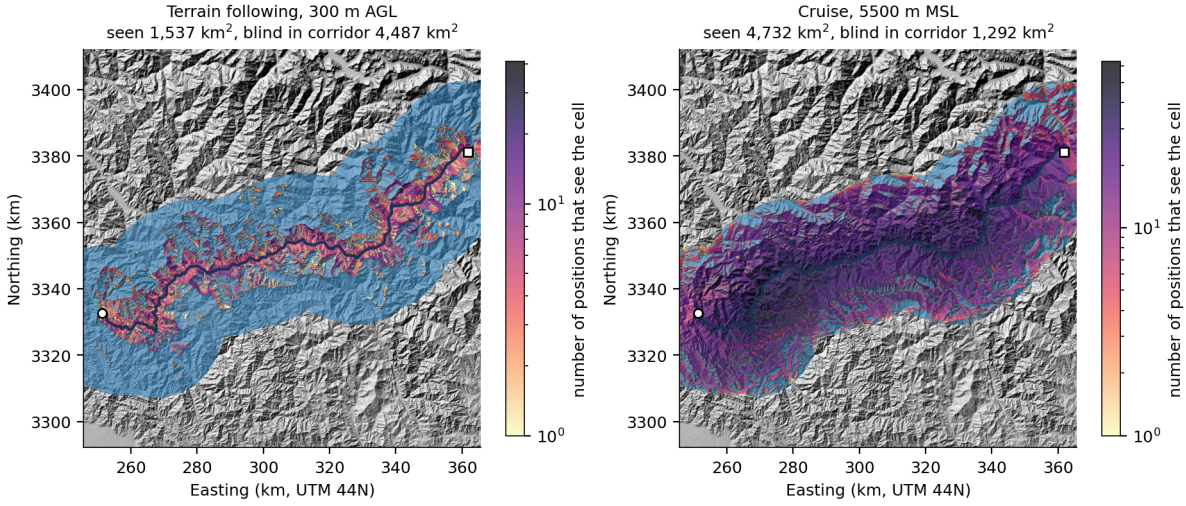

Figure 5. How many positions see each cell (colour) and blind spots in the corridor (blue).

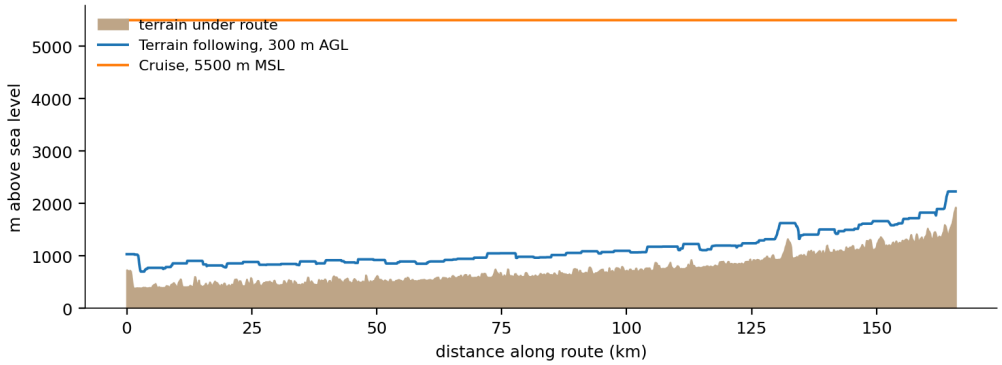

Figure 6. Terrain under the route and the two flight profiles.

> **Finding**
>
> Flying low in the valley sees only **25.5%** of the corridor; the valley walls hide everything behind the first ridge. At 5,500 m the aircraft sees **78.6%**. Low flying is good for staying hidden and poor for surveillance, the trade-off that Track 3 optimises. Each scenario takes about 4.6 s for 167 viewsheds.

4.3 First benchmark: cost against DEM size (F1)

Table 10. Two ways to grow the problem with DEM size (median compute time).

DEM cells	F1: route = 0.8 x DEM width, unlimited radius	Positions	F1b: fixed 12 km route, 20 km radius
250,000	0.07 s	13	0.07 s
1,000,000	0.43 s	25	0.21 s
4,000,000	2.81 s	49	0.31 s
16,000,000	18.78 s	97	0.31 s

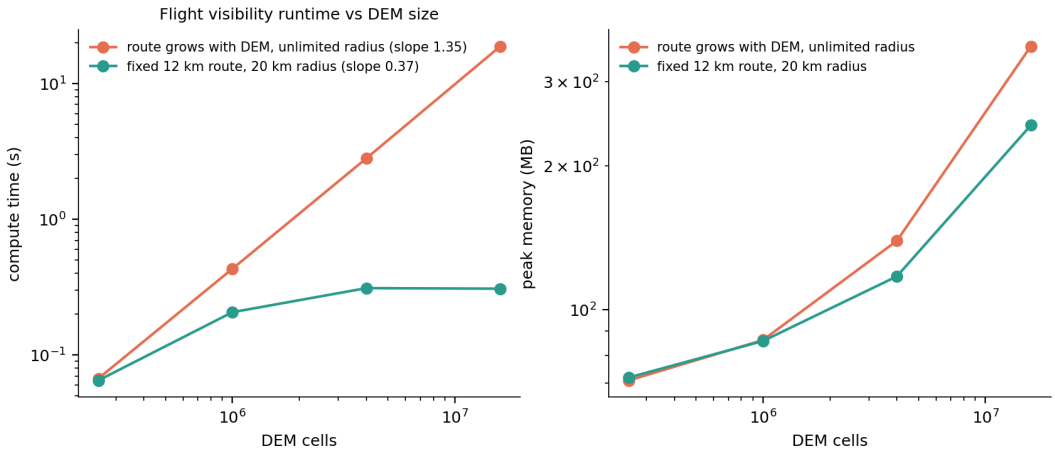

Figure 7. Compute time and memory against DEM size for the two set-ups.

> **Finding**
>
> The work is *positions x cells per viewshed*, not DEM size itself. When route and radius grow with the terrain, each viewshed covers the whole DEM and the number of positions grows with its width, so cost grows like N·√N (measured exponent 1.35). With a fixed route and radius only reading the larger DEM grows (exponent 0.37).

5. Track 3: Optimal Visibility Corridor

PERSON 3

5.1 Objective

Track 3 uses *visibility of the aircraft from ground radars*: find a route between two points that keeps the aircraft hidden from the six Track 1 radars without making it much longer. The aircraft flies 150 m above the ground; terrain where that would exceed a 5,000 m ceiling is blocked.

Exposure of a cell is the sum over the radars that can see an aircraft there of (1 - d / range), with a range of 40 km, so nearer radars count more. The **cost** of moving between neighbouring cells u and v on the 8-connected grid is

$$c(u, v) = L_{uv} \cdot (1 + \alpha \cdot (E_u + E_v) / 2) + \beta \cdot \max(0, z_v - z_u)$$

where L is 30 m or 42.4 m, α weighs exposure against distance and β = 2 penalises climbing. Setting α = β = 0 gives the shortest-distance baseline.

5.2 Solvers

Table 11. Route solvers (corridor/optimizer.py). All exact solvers return identical optimal costs at every size.

Solver	Description	Exact?
Dijkstra (Python)	Binary heap, stops at the goal; the baseline	yes
A* (Python)	Heuristic = straight-line distance + β · height still to climb; admissible and consistent	yes
Dijkstra (SciPy, C)	Explicit sparse graph of all edges; graph build timed separately	yes
Multi-resolution	Search a 4x coarser grid, then A* only within a buffer around that route	approximate

5.3 Scenario: Devprayag to Joshimath

Table 12. Routes between the Track 2 endpoints, scored with the same metrics.

Route	Length	Exposure (weighted km)	Length exposed	Total climb
Shortest distance (α = 0)	130.6 km	31.5	39.4%	26,492 m
Track 2 valley route	177.9 km	26.5	25.9%	19,408 m
Least exposure (α = 10, β = 2)	**170.0 km**	**2.2**	**2.8%**	**21,639 m**

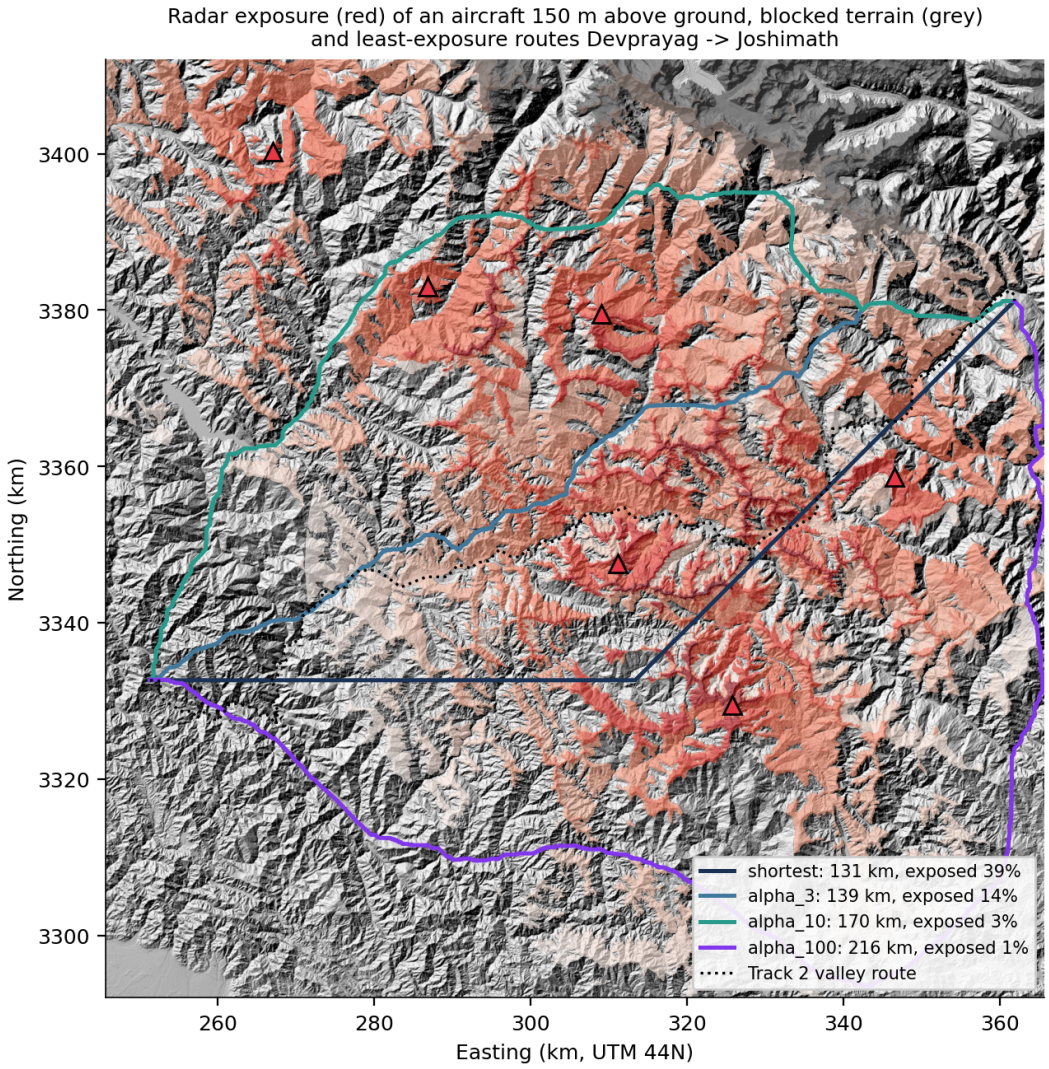

Figure 8. Radar exposure (red), blocked terrain (grey) and routes for several values of α.

Finding

The least-exposure route is **30.2% longer** than the shortest one but cuts the exposure integral by **93.1%**, leaving 2.8% of its length visible to any radar. The Track 2 valley route is already partly masked (25.9% exposed), but the optimiser does far better.

5.4 First benchmark: search time against graph size (C1)

Table 13. Search time including graph build (median), and the cost of the approximate method above the optimum.

Nodes	Dijkstra (Python)	A* (Python)	SciPy (C)	Multi-resolution	Multi-res. cost gap
250,000	0.73 s	0.79 s	0.11 s	0.10 s	0.02%
1,000,000	2.76 s	2.92 s	0.48 s	0.27 s	0.00%
4,000,000	16.07 s	15.09 s	1.97 s	0.93 s	0.08%
16,000,000	70.24 s	54.96 s	8.98 s	3.04 s	1.40%

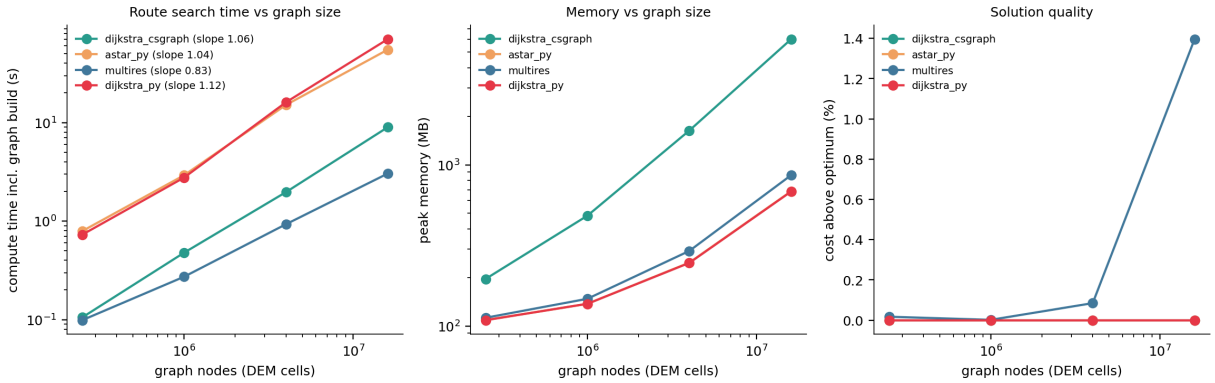

Figure 9. Search time, peak memory and solution quality against graph size.

Finding

All exact solvers scale close to linearly (exponents 1.04 to 1.12; theory N log N). Moving the same algorithm from Python to C gains 8x at 16 M nodes but builds the whole graph in memory: 6,025 MB against 683 MB. Plain A* barely helps here; Appendix A.3 explains why and shows the fix.

6. Progress Against the Plan

The project plan has six phases per track. The mid-submission milestone (about 50%) is phases 0 to 3 plus the first scaling benchmark for every track; the shaded rows below are complete and reported in sections 2 to 5.

Table 14. Status per phase and track. Shaded = mid-submission scope, complete.

Phase	Track 1	Track 2	Track 3
0. Shared data, conventions, protocol	**complete**	**complete**	**complete**
1. Research and design	**complete**	**complete**	**complete**
2. Implementation and validation	**complete**	**complete**	**complete**
3. Visualisation (maps, QGIS project)	**complete**	**complete**	**complete**
4a. Scaling against DEM size	**E1 complete**	**F1 complete**	**C1 complete**
4b. Parameter studies	first results (A.1)	first results (A.2)	first results (A.3)
5. Optimisations	first results (A.1)	first results (A.2)	first results (A.3)
6. Analysis, rendering, final report	planned	planned	planned

Deliverables available now

- Source code for all three tracks with a shared benchmark harness, in the team repository.
- GeoTIFF rasters, GeoJSON routes and CSV metrics for every scenario and benchmark run.
- A styled QGIS project with all outputs grouped by track.
- This report and a slide deck (mid-evaluation section plus backup slides).

7. Future Work

The second half of the project completes phases 4 to 6 for every track. Several of these experiments have first results already (Appendix A); the plan below extends them and addresses the limitations found so far.

7.1 Track 1: radar visibility

- **Radar siting optimisation.** Choose radar locations that maximise joint coverage (greedy selection over candidate ridge tops), instead of the fixed siting rule, and measure how cost grows with the number of candidates.
- **Detection model.** Replace binary visibility with a range-dependent detection probability based on the radar equation, and compare coverage maps.
- **Larger-than-memory terrain.** Tile the DEM so that 8000 x 8000 (64 M cells) and larger grids run within a fixed memory budget, and measure the tiling overhead.
- **Surface against terrain model.** Quantify how trees and buildings in the surface model (DSM) change visibility compared with a bare-earth model (DTM).

7.2 Track 2: flight-route visibility

- **Bounded-memory parallel runs.** Threads already give an 8x speed-up (A.2); stream results in fixed-size batches so memory stays flat as routes grow.
- **Viewpoint selection.** Find the smallest set of positions that reproduces the dense coverage (greedy set cover), and turn the altitude-dependent spacing result into an automatic spacing rule.
- **More routes and aircraft.** Ridge-line and straight routes, several aircraft at once, and time-to-first-sighting maps for surveillance planning.

7.3 Track 3: optimal visibility corridor

- **Done since the mid-submission (Appendix B):** compiled searches and landmark A*, an interactive planner (precomputed fields, alpha bank, parallel rebuilds, anytime search), detection-probability routing with a certified risk budget, a tile overlay, altitude choice, a radar-placement game, alternative corridors and robustness checks.
- **Remaining:** combine the risk budget with uncertain radar positions (a chance constraint over scenarios), and run the exact constrained search on 16 M cells with a stronger bound.
- **Aircraft constraints.** Add turn radius, climb-rate and minimum-clearance limits through a state-expanded graph, so the result is a flyable corridor rather than a grid path.
- **Multi-objective routes.** Compute the full trade-off front between distance, exposure and climb, and k alternative routes rather than a single optimum.
- **Corridor width.** Score a corridor of given width around the route, not only the centre line.

7.4 Across the tracks

- **A second study area.** Repeat the scaling benchmarks on a flatter region (for example the Indo-Gangetic plain) to separate the effect of size from that of terrain ruggedness.
- **Operating-system effects.** Repeat the parallel experiments on Linux, where worker processes start far faster than on Windows.

- **Interactive rendering.** Measure pan and zoom responsiveness in the QGIS desktop as well as off-screen render time.
- **Final integration.** One consolidated comparison of the three workloads, the final report and the final presentation.

7.5 Timeline to the final submission

Table 15. Plan for the second half of the project.

Period	Track 1	Track 2	Track 3	Team
Week 1	radar siting; detection model	bounded-memory threads	compiled landmark A*	second study area data
Week 2	tiling for 64 M cells	viewpoint selection; spacing rule	aircraft constraints	Linux benchmark runs
Week 3	DSM vs DTM; full parameter study	more routes; first-sighting maps	trade-off front; alternatives	interactive rendering tests
Week 4	analysis and write-up	analysis and write-up	analysis and write-up	final report and presentation

8. Conclusions

- All three applications run end to end on one shared, documented data set and pass their correctness checks; the production viewshed agrees with the exact line-of-sight answer on 99.8% of cells.
- Algorithm choice dominates cost: the exact viewshed grows as $N^{1.5}$ and would take about 68 minutes at 16 M cells, while linear methods take well under a second.
- Each workload has its own cost driver: the area one radar covers (Track 1), the number of positions times the area each sees (Track 2), and the size of the route graph (Track 3).
- The applications answer real questions: six radars see 16.7% of the ground, valley flying sees 25.5% of its corridor, and an optimised route cuts radar exposure by 93.1% for 30.2% more distance.
- The mid-submission scope is complete, and the experiments for the second half are already under way.

References

European Space Agency (2021). Copernicus DEM: Global and European Digital Elevation Model (GLO-30). Copernicus Programme.

Wang, J., Robinson, G. J. and White, K. (2000). Generating viewsheds without using sightlines. *Photogrammetric Engineering & Remote Sensing* 66(1), 87-90.

Franklin, W. R. and Ray, C. K. (1994). Higher isn't necessarily better: visibility algorithms and experiments. *Advances in GIS Research*, 751-770.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik* 1, 269-271.

Hart, P. E., Nilsson, N. J. and Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics* 4(2), 100-107.

Goldberg, A. V. and Harrelson, C. (2005). Computing the shortest path: A* search meets graph theory. *Proc. ACM-SIAM SODA*, 156-165.

Meyer, U. and Sanders, P. (2003). Delta-stepping: a parallelizable shortest path algorithm. *Journal of Algorithms* 49(1), 114-152.

Koenig, S. and Likhachev, M. (2002). D* Lite. *Proc. AAAI*, 476-483.

Ramalingam, G. and Reps, T. (1996). An incremental algorithm for a generalization of the shortest-path problem. *Journal of Algorithms* 21(2), 267-305.

Juttner, A., Szviatovszki, B., Mecs, I. and Rajko, Z. (2001). Lagrange relaxation based method for the QoS routing problem. *Proc. IEEE INFOCOM*, 859-868.

Carlyle, W. M., Royset, J. O. and Wood, R. K. (2008). Lagrangian relaxation and enumeration for solving constrained shortest-path problems. *Networks* 52(4), 256-270.

Delling, D., Goldberg, A. V., Pajor, T. and Werneck, R. F. (2011). Customizable route planning. *Proc. SEA*, LNCS 6630, 376-387.

Richter, S., Thayer, J. T. and Ruml, W. (2010). The joy of forgetting: faster anytime search via restarting. *Proc. ICAPS*, 137-144.

GDAL/OGR contributors (2025). GDAL Geospatial Data Abstraction Library, version 3.13. Open Source Geospatial Foundation.

QGIS Development Team (2025). QGIS Geographic Information System, version 3.44. Open Source Geospatial Foundation.

Appendix A. Results Beyond the Mid-Submission Scope

These experiments belong to phases 4b and 5 of the plan. They have been run with the same protocol and are reported here for completeness; Section 7 describes how they will be extended.

A.1 Track 1: parameters and optimisations

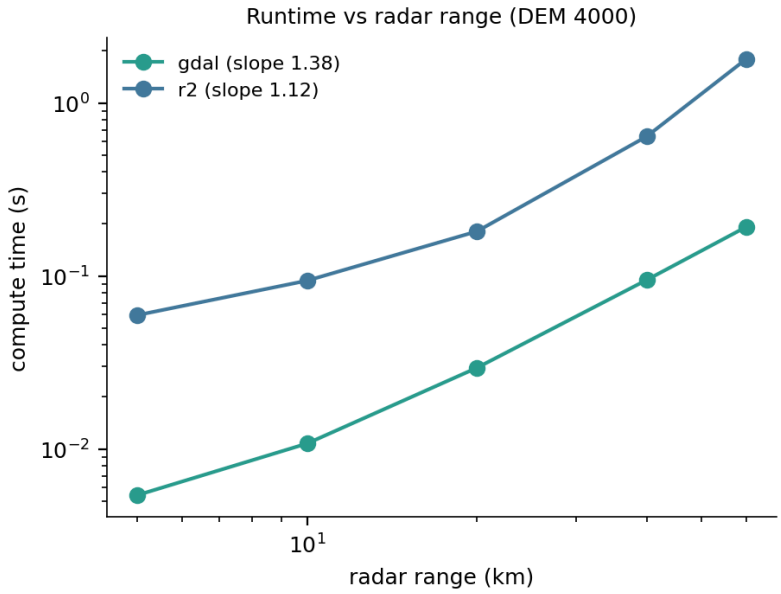

Figure 10. Compute time against radar range on the 4000 DEM (E2).

Range (E2). Work grows with the area inside the range: compute-time exponents 1.38 (GDAL) and 1.12 (R2) against range. **Mast height (E3)** changes coverage but not cost.

Table 16. Region clipping (E5): reading only the window that the radar range covers (GDAL, DEM 4000).

Range	Full DEM read: runtime / peak	Window read: runtime / peak
10 km	0.348 s / 242 MB	0.046 s / 73 MB
20 km	0.352 s / 242 MB	0.089 s / 103 MB
40 km	0.439 s / 262 MB	0.294 s / 220 MB

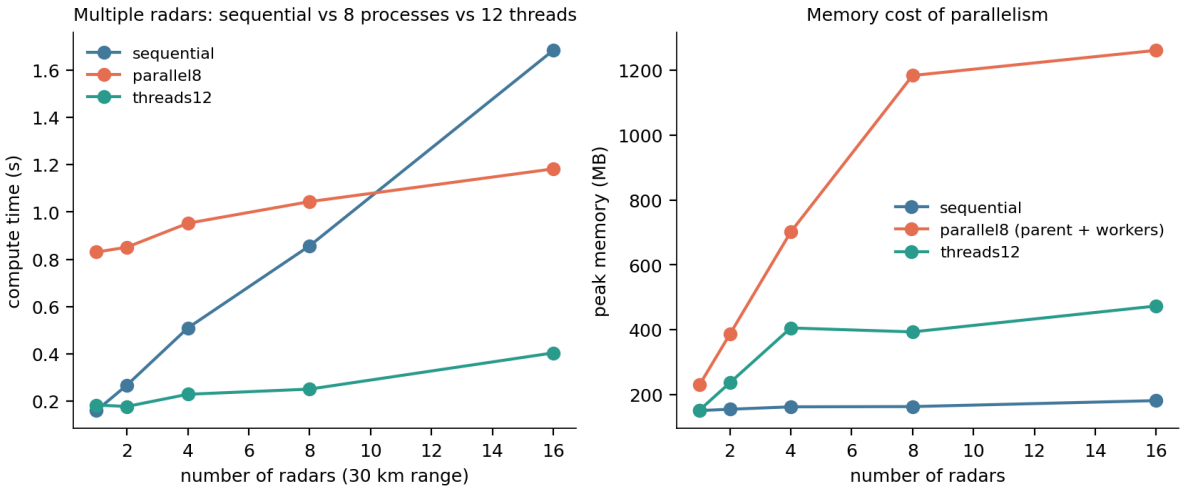

Figure 11. Several radars (E4): sequential, 8 worker processes and 12 threads.

Optimisation: threads instead of processes

On Windows every worker process starts from scratch (about 0.8 s, longer than one 30 km viewshed) and copies results back, so 8 processes only helped from 16 radars (1.42x). GDAL releases Python's global lock inside the viewshed, so 12 threads run truly in parallel: 1.51x at 2 radars and 4.16x at 16 (1.68 s to 0.40 s), at about 0.5 GB because the windows are held together.

Table 17. Coarser DEMs (E6) against the 30 m result. Raw agreement stays high because most cells are hidden either way; kappa shows the real loss.

Cell size	Compute time	Agreement	Kappa
30 m	0.099 s	100.00%	1.000
60 m	0.030 s	99.98%	0.898
90 m	0.016 s	99.97%	0.835
120 m	0.010 s	99.96%	0.777

A.2 Track 2: parameters and optimisations

Table 18. Sample spacing (F2). Runtime is linear in the number of positions (exponent 1.05).

Spacing	Positions	Compute	Overlap with 250 m run	Missed area
250 m	665	17.31 s	100.0%	0 km²
500 m	333	8.69 s	98.3%	27 km²
1000 m	167	4.13 s	94.7%	86 km²
2000 m	84	1.97 s	86.3%	223 km²
4000 m	43	0.98 s	72.3%	449 km²

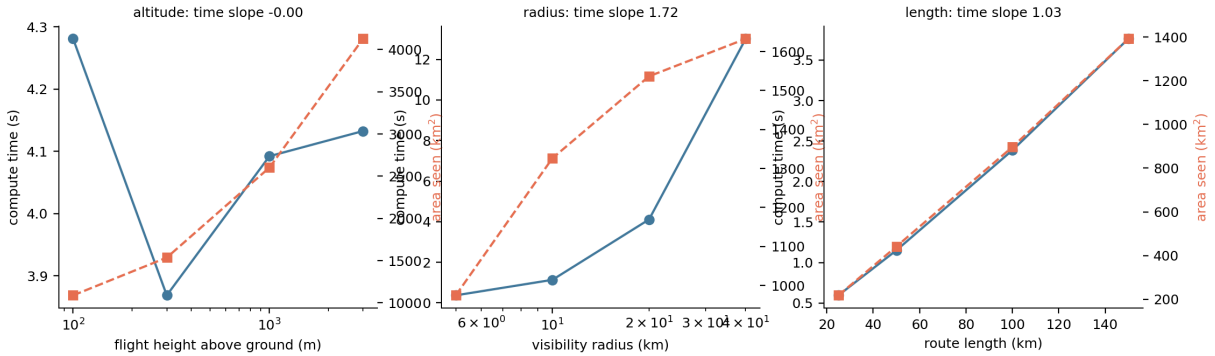

Figure 12. Flight height (F3), sensor radius (F4) and route length (F5).

Runtime is roughly quadratic in the sensor radius (exponent 1.72), linear in route length and independent of flight height, while the area seen rises with both radius and height.

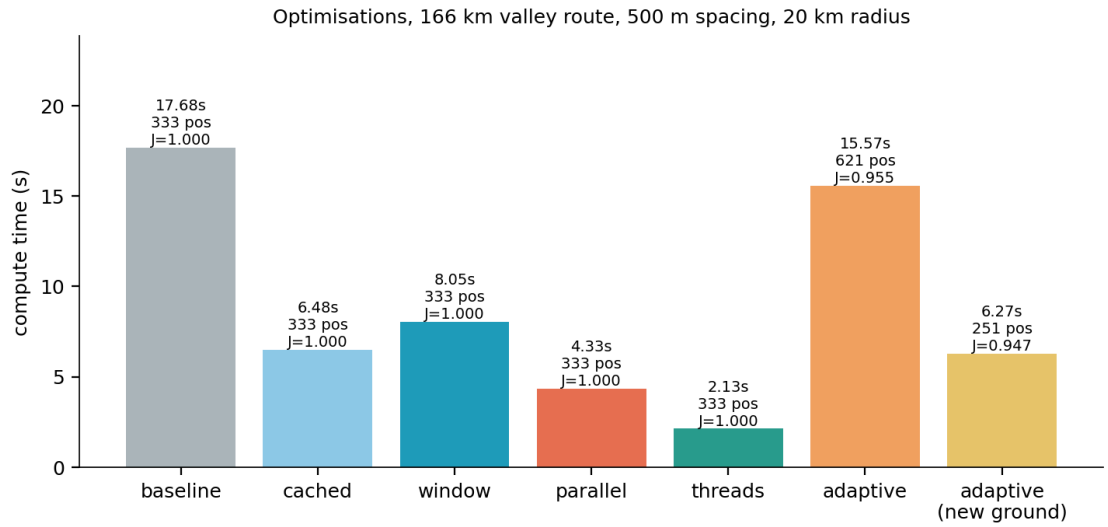

Figure 13. Execution modes on the 166 km route with 500 m spacing (F6).

Table 19. Optimisations (F6). Baseline re-opens the DEM for every position.

Mode	Compute	Positions	Peak memory	Coverage vs dense (Jaccard)
baseline	17.68 s	333	244 MB	1.000
cached	6.48 s	333	309 MB	1.000
window	8.05 s	333	243 MB	1.000
parallel	4.33 s	333	750 MB	1.000
threads	2.13 s	333	684 MB	1.000
adaptive	15.57 s	621	1,210 MB	0.955
adaptive gain	6.27 s	251	615 MB	0.947

Optimisation: threads; sampling should follow altitude

Keeping the DEM in memory gives 2.7x; 12 threads give 8.3x with identical results. Adaptive sampling did not pay off: refining where neighbouring views differ refines almost everywhere in a winding valley (621 positions), and refining only where a position sees new ground stays on or below the fixed-spacing curve (Figure below). The useful lever is altitude: at cruise, 4 km spacing keeps 97.5% of the coverage with 43

viewsheds; valley flying needs 500 m.

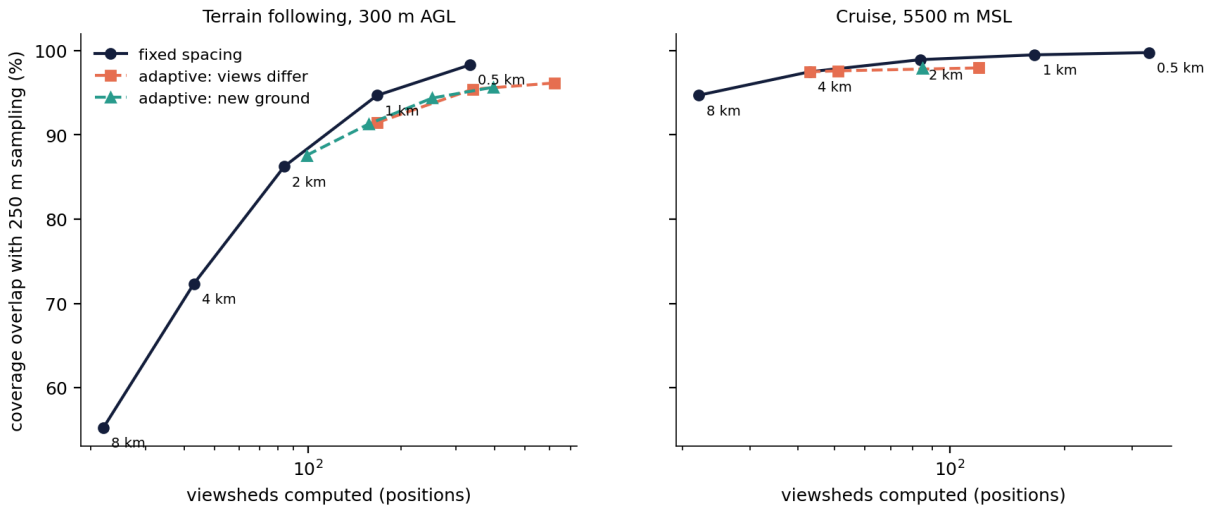

Figure 14. Positions against coverage for fixed spacing and two adaptive criteria, at valley and cruise altitude.

A.3 Track 3: parameters and optimisations

Why plain A* did not help, and the fix

A* expands fewer nodes only when its estimate of the remaining cost is close to the truth. The straight-line estimate assumes 1 cost unit per metre, but exposed metres cost 5 to 17, so with α between 1 and 100 plain A* expanded 64.4% to 86.0% of the 4 M nodes. **Landmark A* (ALT)** precomputes exact costs to and from four landmarks once; the triangle inequality then gives a much tighter estimate that is still admissible. It returns the same optimum while expanding 3.5% to 14.5% of the nodes at every α. On the 16 M-node scenario: 4,555,338 nodes in 21.5 s instead of 14,335,945 nodes in 67.3 s, after 36.9 s of one-off preprocessing.

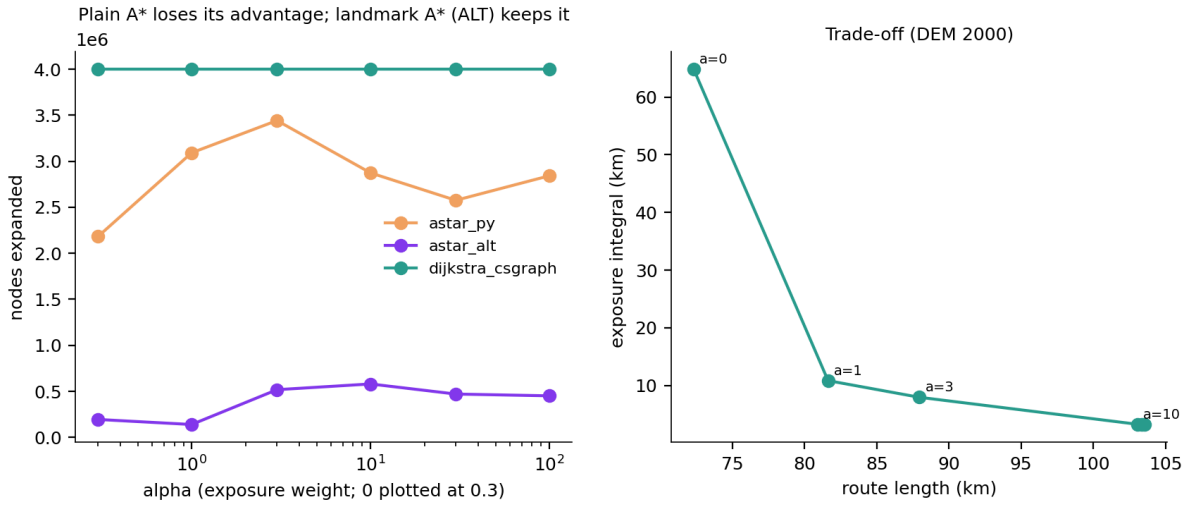

Figure 15. Nodes expanded against α for plain A*, landmark A* and Dijkstra, and the length-exposure trade-off (DEM 2000).

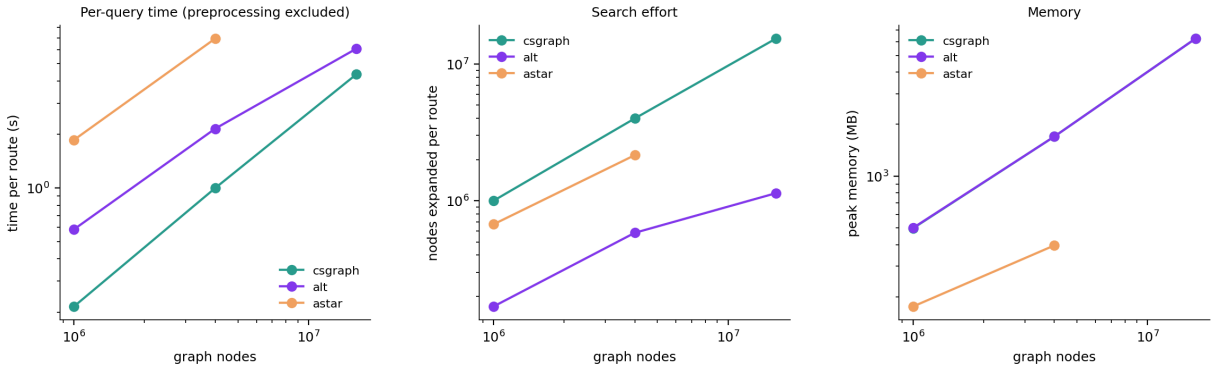

Figure 16. Eight routes on the same map (C6): time per route, nodes expanded and peak memory.

Many routes on one map (C6). Per route, landmark A* is about 3x faster than plain A*, but in Python it does not yet beat the C Dijkstra (6.08 s against 4.36 s at 16 M nodes) even though it expands only 7.1% of the nodes; a compiled version is planned (Section 7.3). All methods return identical costs.

Table 20. Multi-resolution on the 16 M-node graph (C3). The exact C Dijkstra takes 8.98 s and 6 GB.

Coarsening factor	Compute	Nodes expanded	Cost above optimum
2	7.70 s	2,255,759	0.04%
4	2.06 s	672,804	1.40%
8	1.17 s	390,980	1.56%

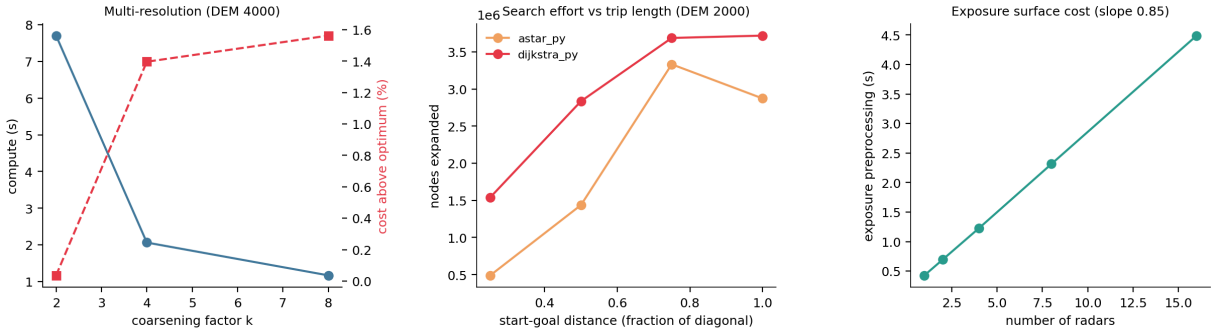

Figure 17. Coarsening factor (C3), trip length (C4) and exposure preprocessing cost (C5).

A.4 Visualisation performance

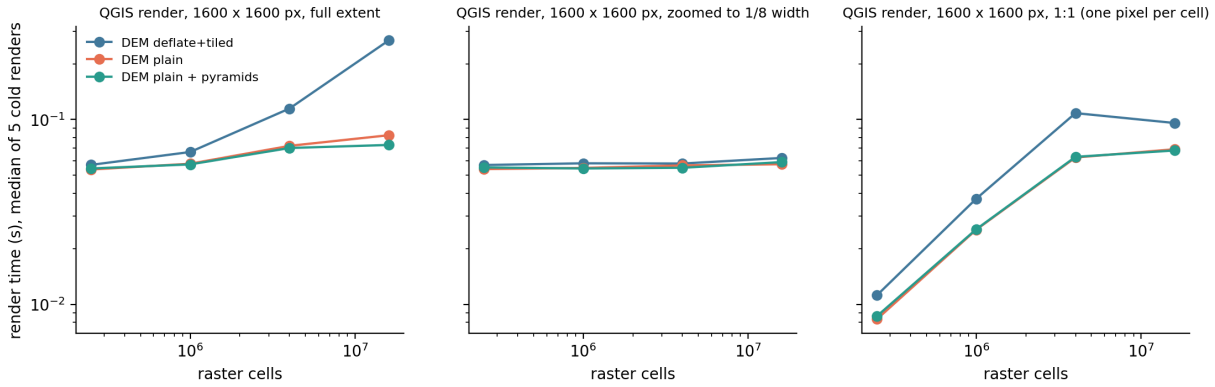

Figure 18. QGIS render time at 1600 x 1600 px (median of five cold renders) for the DEM stored three ways.

Plain or pyramided rasters draw in 55-80 ms at every size: QGIS reads only what fits on screen, so cost

follows the screen, not the raster. Compressed, tiled GeoTIFFs without pyramids are the exception at full extent (0.27 s at 16 M cells), because every tile must be decompressed, and took 19.9 s when GDAL's cache was limited to 1 MB. Recommendation: build pyramids for compressed outputs.

A.5 Comparison across the tracks

Table 21. What drives the cost of each workload. Raw runtimes across tracks are not a fair contest because the tracks do different work.

Track	Unit of work	Cost grows with	At 16 M cells	Most effective optimisation
1 Radar	one viewshed	cells in range (range²)	0.10 s, 242 MB	read only the range window; threads
2 Flight	one viewshed per position	positions x radius²	18.8 s (97 positions)	threads; spacing set by altitude
3 Corridor	graph search on exposure	nodes (N log N)	9-70 s, 0.7-6 GB	C code, landmark A*, multi-resolution

Limitations

- Larger DEMs are also more rugged, which mixes size and terrain effects (Section 7.4).
- Peak memory is the Windows peak working set; worker processes are reported separately from the parent.
- The Copernicus model is a surface model, so trees and buildings block sight lines.
- Exposure (1 - d/range) is a simple stand-in for the radar equation, and the aircraft is a point at a fixed height above ground.

Appendix B. Track 3: Interactive Planning

PERSON 3 · WORK AFTER THE MID-SUBMISSION

A route planner is only useful if it answers while the user is still looking at the map. This appendix answers the question raised in our project meeting: *what should be precomputed, what should be computed on the fly, which algorithms keep panning, zooming and editing smooth, and what to do when the precomputed data no longer applies?* It then reports further extensions of the corridor model. Every number comes from the 120 x 120 km, 16 M-cell DEM unless stated otherwise, and every route is checked against an exact search.

B.1 Compiled search

The Python searches of Section 5 were rewritten in C on the implicit grid (no stored graph): Dijkstra, A* and landmark A* (ALT) (corridor/c/gridsearch.c, called through ctypes). On 16 M nodes one exact route takes 2.51 s with 635 MB (SciPy: 8.98 s, 6,025 MB) and landmark A* 0.37 s after 3.64 s of landmark preprocessing on 8 threads. All costs are identical to the optimum.

B.2 What to precompute: a ladder of levels

Table 22. Precompute against query time (E1, mean of 20 routes, 16 M cells; every route exact).

Level	Precompute	Memory	Route for a new start	Route for a new goal
0. Nothing (C Dijkstra)	-	-	1,488 ms	1,812 ms
0. A* (straight line + climb)	-	-	772 ms	993 ms
1. Landmarks (4), ALT	3.7 s	488 MB	212 ms	274 ms
1. Landmarks (8), ALT	7.6 s	977 MB	212 ms	243 ms
2. Cost-to-go field to the goal	**3.0 s (0.31 s parallel)**	**183 MB**	**0.06 ms (lookup)**	**one new field**

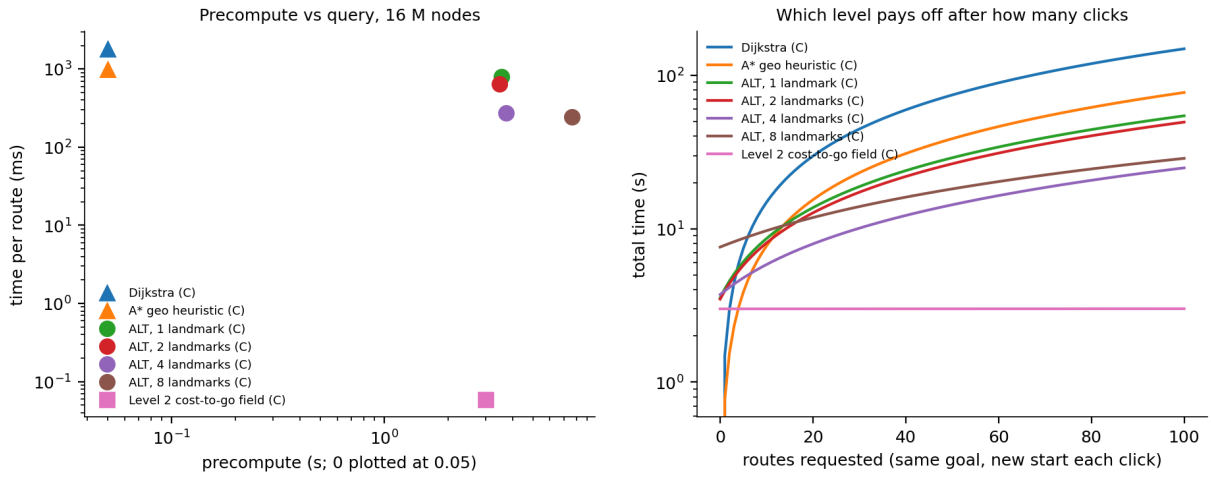

Figure 19. Left: precompute against time per route. Right: total time after n clicks with the same goal.

Finding: precompute the field for the current goal

A **cost-to-go field** stores, for every cell, the cost of the best route to the goal and the next step. One backward search builds it; afterwards a route from *any* start is read off in 0.06 ms with no search, exact. It pays off from the second click. Landmarks are the right precompute when the goal changes often; four are

the sweet spot (eight double memory for 11% faster queries).

B.3 Building the field on 12 cores

Dijkstra settles one node at a time and cannot use more cores. **Delta-stepping** (Meyer and Sanders 2003) relaxes a whole bucket of nodes with similar cost at once; lock-free atomic updates keep it exact (corridor/c/parallel.inc). The OpenMP barrier of the MinGW runtime sleeps between phases and made the parallel version slower than one core; replacing it with a spinning barrier fixed this. On 16 M cells the field takes 2.98 s with Dijkstra, 1.60 s with delta-stepping on one core and **0.31 s on 16 threads (9.6x)**, identical to the last bit. The alpha bank of five fields (B.5) now builds in 1.9 s.

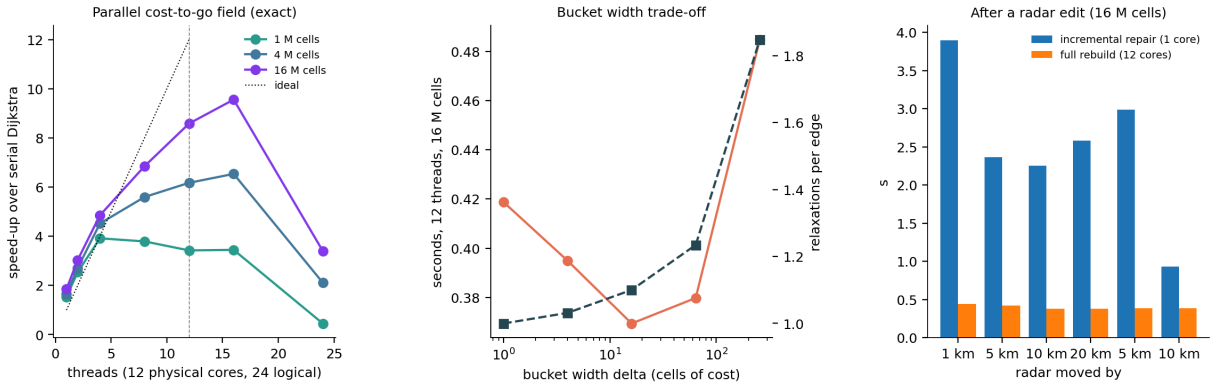

Figure 20. Speed-up against threads (12 physical cores), bucket width, and repair against a parallel rebuild after radar edits.

B.4 Edits: moving the start, a radar, the goal

Table 23. Medians over the edits of E3 and E8. Every repaired field equals a fresh one (max difference below 1e-9).

Event (16 M cells)	Incremental method	Time	From scratch	Time
start moved (click)	field lookup	0.09 ms	C Dijkstra	2,698 ms
start moved (click)	D* Lite replan	62 ms		
radar moved	threat surface of that radar only	191 ms	all six viewsheds	~1,000 ms
radar moved	field repair (dynamic SSSP)	2,581 ms	parallel field rebuild	386 ms
radar moved	D* Lite update	1,590 ms		

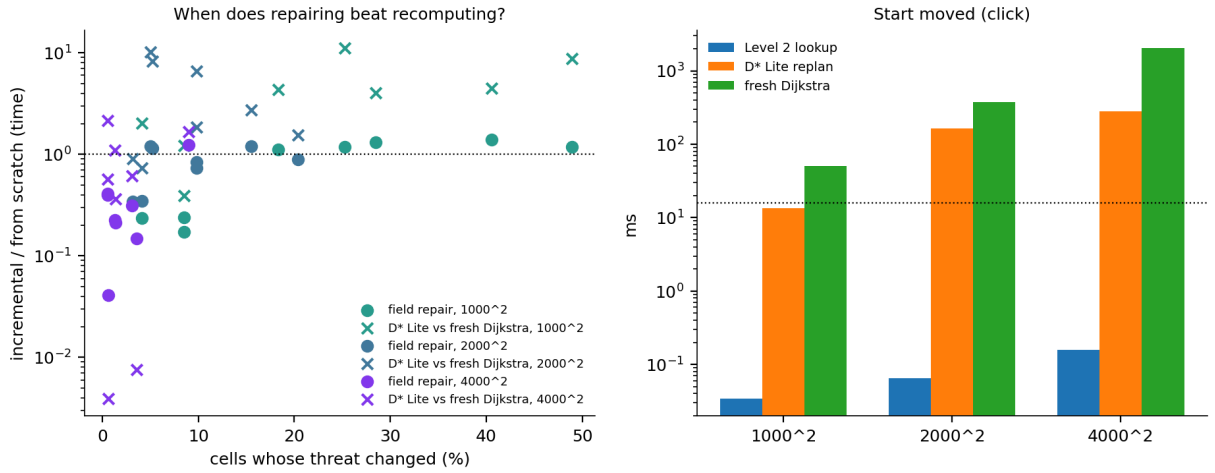

Figure 21. Left: incremental time / from-scratch time against the share of cells whose threat changed. Right: moving the start.

> **Finding: repair locally, but rebuild in parallel**
>
> Only the radar that moved is recomputed (one viewshed window), never the whole threat surface. For the route, the incremental field repair and D* Lite are exact but their cost depends on how far the change propagates: a radar has a 40 km reach, so its edit touches a large share of the routes. With 12 cores a full parallel rebuild (386 ms) beat the repair in every edit measured. D* Lite shines only when the start moves along its route (often no work at all), which the field lookup already does in microseconds.

B.5 When the precompute stops being valid: changing alpha

The exposure weight alpha changes every edge cost, so fields and landmarks are invalid. Two remedies. **Anytime coarse-to-fine:** exact routes on a 16x and 4x coarser copy of the grid, then a full-resolution search inside a corridor around them, then the exact field; each stage is drawn as it arrives: first route after 11 ms (12.5% above the optimum), within 0.17% after 0.52 s. Restarting weighted A*, the textbook anytime method, was slower at every quality level (Figure). **Alpha bank:** precompute the next-step fields for a ladder of alpha values (64 MB each); the slider then shows the exact route for a banked alpha at once and the nearest banked route in between.

Table 24. Alpha bank (E7), 10 starts per alpha; a lookup takes about 1 ms. Bank of 5: 305 MB; of 11: 671 MB.

alpha requested	5 banked (2-50): used	cost above exact	11 banked (1-100): used	cost above exact
1	2	0.36% (max 1.14%)	1	0.00%
2	2	0.00% (max 0.00%)	2	0.00%
3	2	0.33% (max 0.93%)	3	0.00%
5	5	0.00% (max 0.00%)	5	0.00%
7	5	0.05% (max 0.13%)	7	0.00%
10	10	0.00% (max 0.00%)	10	0.00%
14	10	0.43% (max 1.27%)	14	0.00%
20	20	0.00% (max 0.00%)	20	0.00%
30	20	1.20% (max 3.43%)	30	0.00%
50	50	0.00% (max 0.00%)	50	0.00%
70	50	0.04% (max 0.10%)	50	0.04%
100	50	0.46% (max 1.02%)	100	0.00%

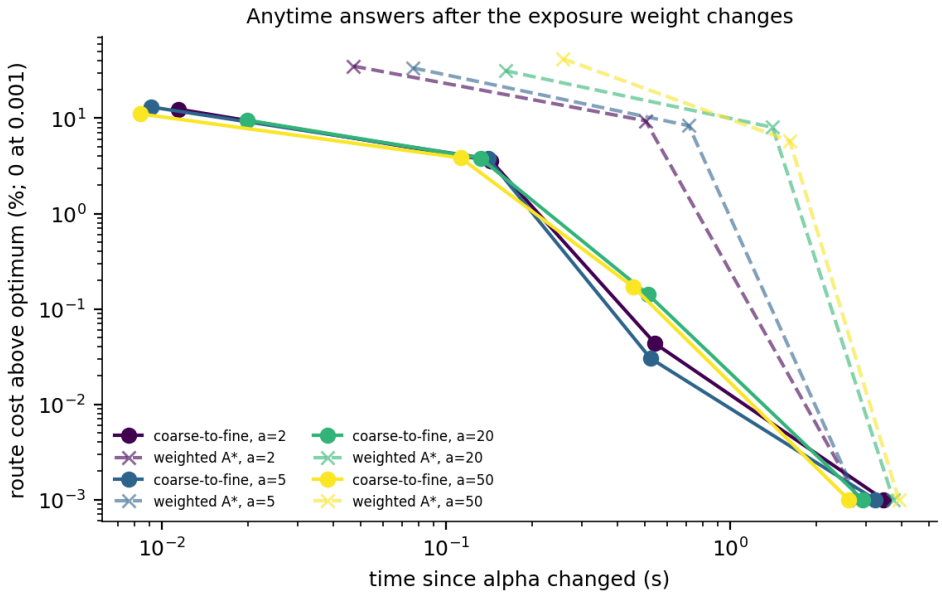

Figure 22. Cost above the optimum against time after alpha changes: coarse-to-fine (circles) and weighted A* (crosses).

B.6 Panning and zooming

A route is one global object, so it should not be re-solved per view. Re-solving the visible window on a coarse-to-fine pyramid took 153 ms per view and often drew a route in a different valley (up to hundreds of pixels off). Drawing the route already known, decimated to about one point per screen pixel, takes 0.03 ms at any zoom: pan and zoom never search. The pyramid's job is the anytime preview of B.5.

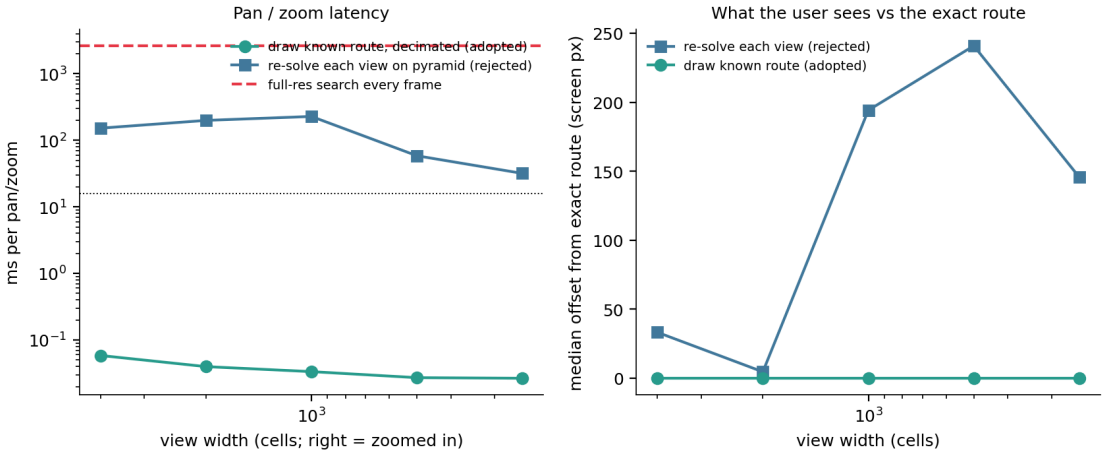

Figure 23. Pan and zoom: latency and on-screen error of re-solving each view (rejected) against drawing the known route (adopted).

B.7 Latency budget

Table 25. What the interactive viewer (corridor/interactive.py) does for each action, 16 M cells.

Interaction	Strategy	Latency	Class
click a new start	field lookup	0.09 ms	within a frame (16 ms)
move the alpha slider (banked value)	alpha bank lookup	~1 ms	within a frame

move the alpha slider (other value)	nearest bank route, then coarse-to-fine, then exact field	1 ms / 523 ms / 311 ms	frame, then responsive
drag a radar	one viewshed + parallel field rebuild	577 ms	responsive (< 1 s)
new goal	landmark A* now, field in the background	274 ms	responsive
pan / zoom	draw the known route	0.03 ms	within a frame

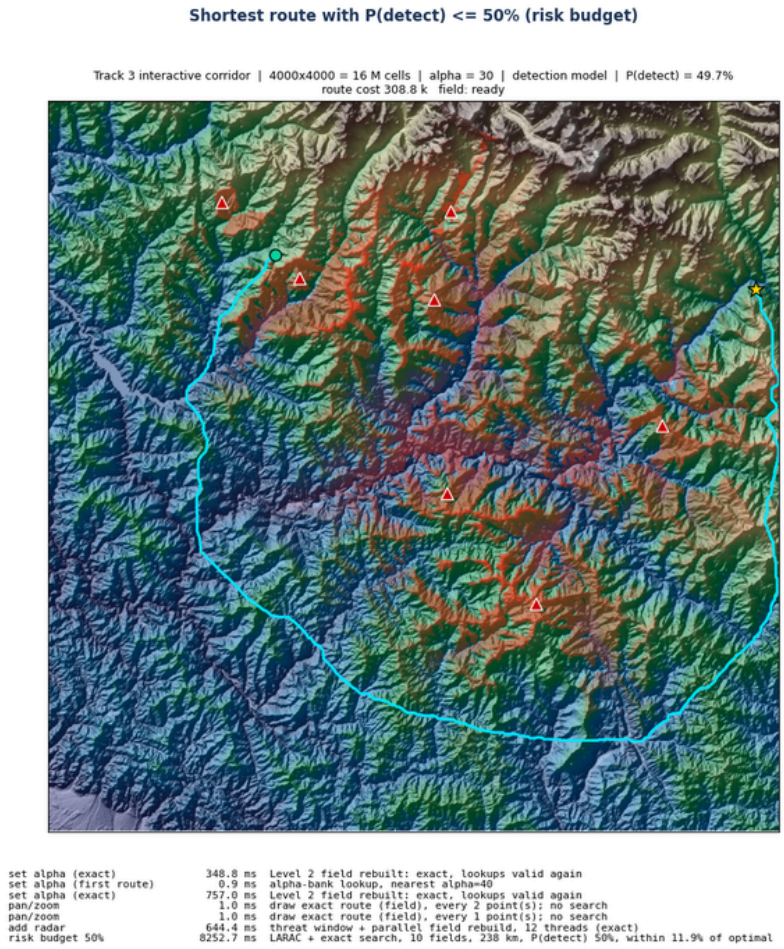

Figure 24. The interactive viewer after a scripted session (clicks, radar drag, alpha change, zoom, risk budget); the panel logs what answered each action.

B.8 From exposure to detection probability

Exposure is a unitless score. The detection model gives each radar that sees the aircraft a probability of detecting it per kilometre flown, $p(d) = 0.5 / (1 + (d / 20\ \text{km})^4)$, so P(detected) = 1 - exp(-Σ length · hazard) with hazard H = Σ -ln(1 - p). The sum is additive along the route, so every solver above applies unchanged.

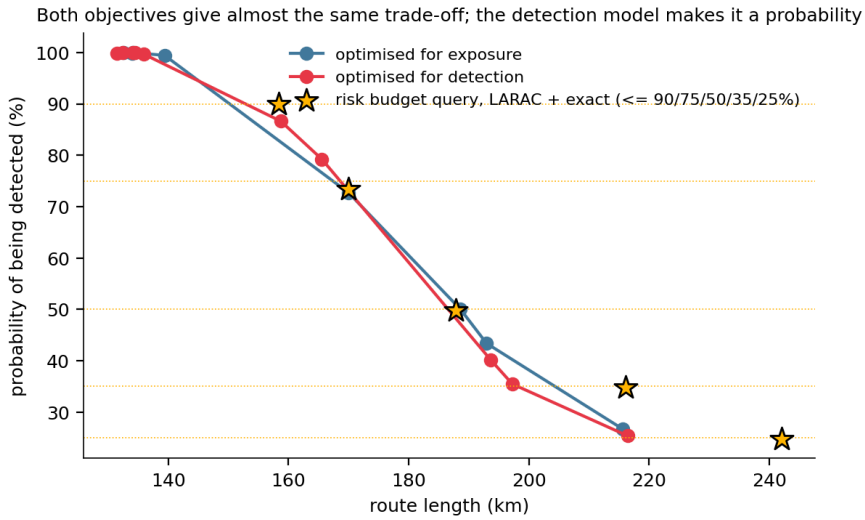

Figure 25. Detection probability against route length for routes optimised for exposure and for detection, and risk-budget answers.

Risk budget: the shortest route with P(detected) ≤ p. This is a resource-constrained shortest path (NP-hard). LARAC (Lagrangian relaxation) moves the price of risk to its best value in a few exact searches (one parallel field each) and certifies a lower bound on the best possible length; an exact label-setting search in a corridor around its routes, pruned with exact bounds and the Lagrangian bound (Carlyle et al. 2008), then closes the gap. On 1 M cells the corridor answer equals the whole-map optimum in all 4 budgets and the remaining gap is the looseness of the bound itself. On 16 M cells LARAC answers in 3.7-5.8 s instead of 23-27 s for bisection.

Table 26. Risk-budget queries on 16 M cells (E9). Certified gap = how far the length can at most be from the best possible; splice = exact search on the union of the LARAC routes when the corridor search hits its label cap.

Budget P(detect) ≤	Exact step	Found	Length	Certified gap	Time
90%	ok	90.0%	158 km	1.1%	4.3 s
75%	no better route in corridor	73.5%	170 km	0.6%	5.3 s
50%	ok	49.8%	188 km	0.3%	4.6 s
35%	splice: ok	34.8%	216 km	9.1%	13.3 s
25%	splice: ok	24.8%	242 km	8.8%	11.3 s

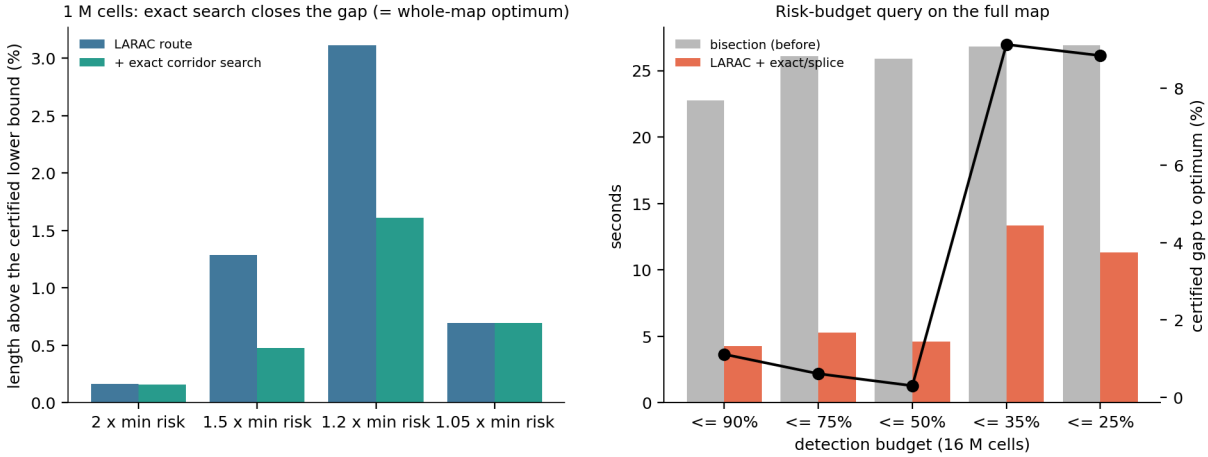

Figure 26. Left: LARAC alone and with the exact corridor search, 1 M cells. Right: time and certified gap on 16 M cells.

B.9 A precompute that survives edits: tile overlay

The field serves one goal; landmarks serve any goal but must be rebuilt after every radar edit (5.0 s). A tile overlay (the idea of customizable route planning) cuts the map into 250 x 250 tiles and precomputes the cost between *portal* cells on each tile's border. Any route is then a search on 14,653 portals instead of 16 M cells, and a radar edit recomputes only the tiles it touches. Portals at regular spacing force detours; adding a portal at every valley floor along a border, where hidden routes actually cross, cut the error several-fold for the same number of portals.

Table 27. Tile overlay, 20 random start-goal pairs (E10). Exact landmark A*: 187 ms per pair.

Portals	Overlay nodes	Precompute	Query + unpack	Cost above exact (mean / max)
every 25 cells	9,374	7.5 s	18 + 52 ms	1.55% / 3.54%
every 10 cells	23,382	19.3 s	24 + 50 ms	0.38% / 0.72%
every 25 cells + valleys	14,653	12.0 s	19 + 52 ms	0.61% / 2.60%
every 50 cells + valleys	12,435	10.2 s	19 + 50 ms	0.78% / 1.28%

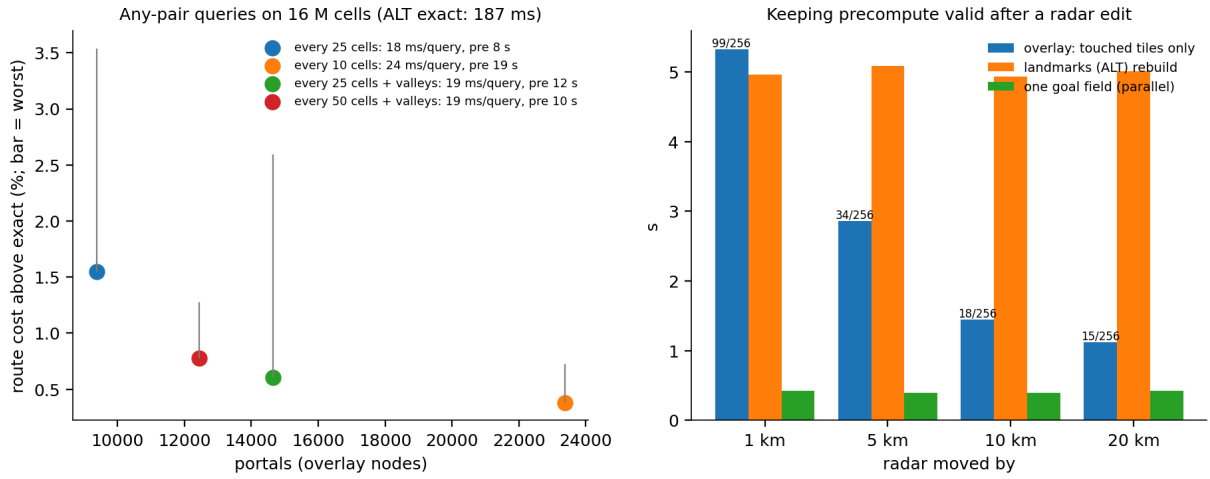

Figure 27. Left: accuracy against overlay size. Right: time to make the precompute valid again after a radar edit.

B.10 Altitude as a decision

The aircraft may fly at 75, 150, 300 or 600 m above ground and change height; each height has its own threat surface (viewsheds depend on the target height) and ceiling mask, flying low costs extra per metre (gamma: terrain-following risk) and every climb costs. The search runs on height levels x cells (corridor/c/layers.inc): 64 M nodes for the 16 M-cell map in 22 s and 732 MB. On the 60 km map a fixed 150 m flight is detected with probability 88.8%; choosing the height freely gives 53.2% without a low-flying penalty (the whole route at 75 m) and 73.3% with it. On the 16 M-cell scenario: 84.6% against 86.7% at a fixed 150 m.

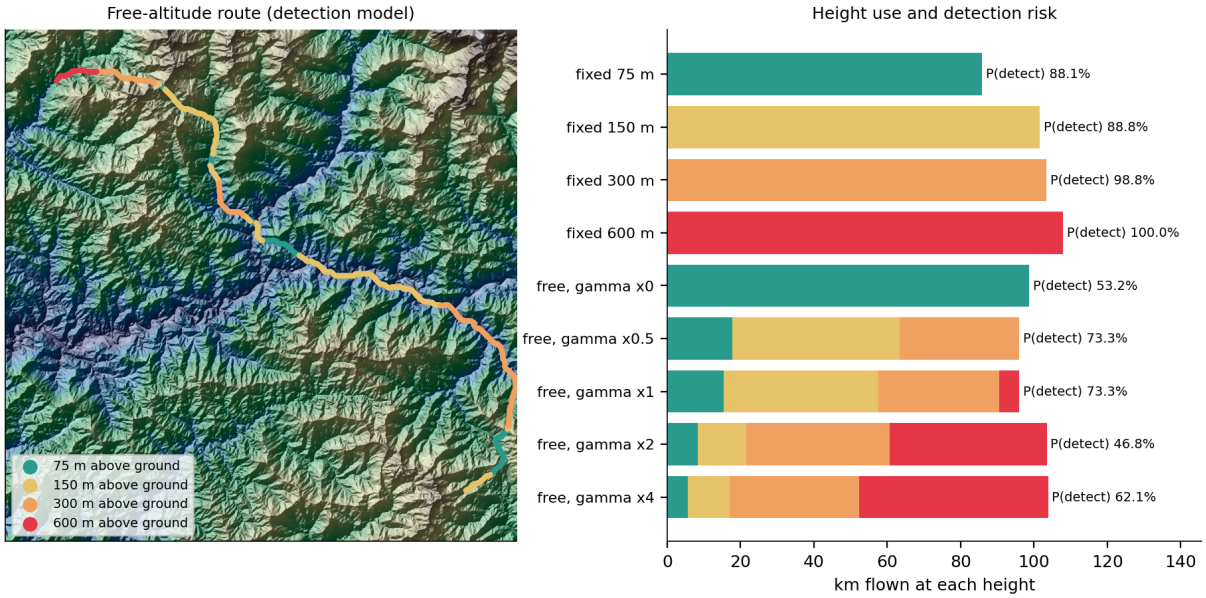

Figure 28. Free-altitude route coloured by height, and height use with detection probability for fixed and free heights.

B.11 Where should the radars go? A placement game

The defender places 6 radars among 40 peaks; the attacker knows them, picks the safest of three entry points and flies the least-risk route. Each attacker reply is one parallel field (about 0.1 s on 4 M cells), so a greedy defender can afford 265 replies. Placing for routes beats placing for coverage: one routing-aware radar already forces a detection probability of 77.6% and two reach 98.2%, while the classic coverage placement (most ground seen) reaches 16.5% with two and 99.7% with all six. The hand-placed scenario radars score 87.8%, random placements a median of 54.6%.

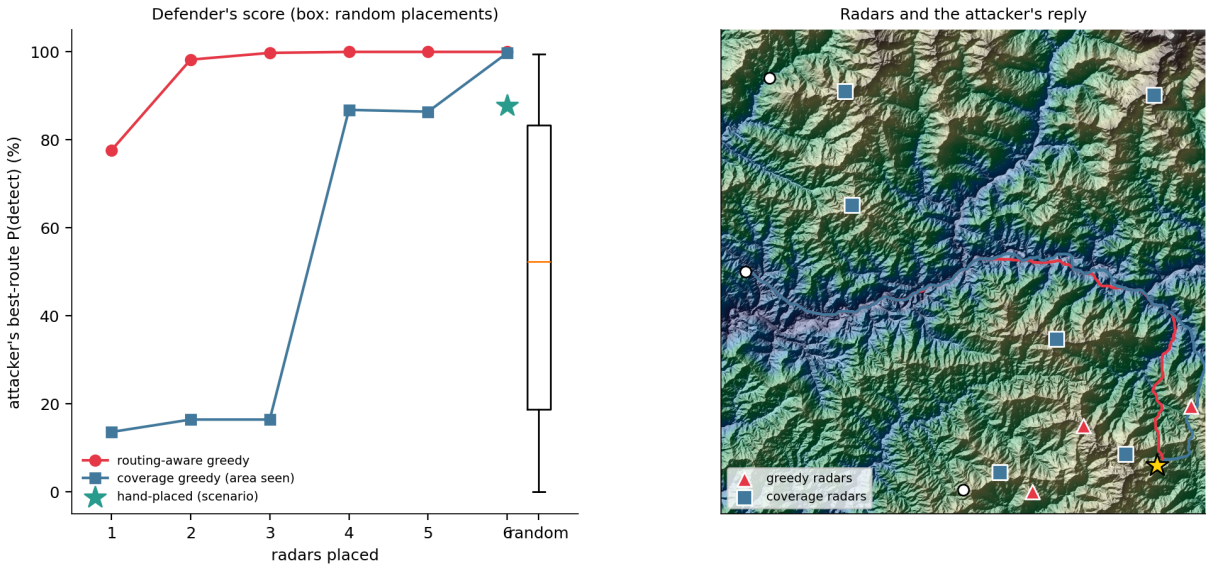

Figure 29. Attacker's best-route detection probability against radars placed, and the radars with the attacker's reply.

B.12 Robustness: uncertain radars, alternatives, model checks

Uncertain radar positions (E13). If each radar is only known to within σ, sampled position scenarios give sampled threat surfaces, and planning on their per-cell mean (expected risk) or on a high per-cell quantile (pessimistic envelope) is still an ordinary shortest path. Judged on unseen scenarios by the planner's own objective (length + α x risk): with σ = 1 km the mean-threat plan is best (129k against 138k for the nominal

plan); with σ = 3 km the pessimistic plan is best on average and in bad cases (138k / 163k against 155k / 193k at the 90th percentile). Note that a better objective is not always a lower detection probability: averaging makes each radar look weaker but wider, and the planner trades that for distance (96 km at 72.9% mean detection against 103 km at 60.4%). A probability target needs the risk budget of B.8.

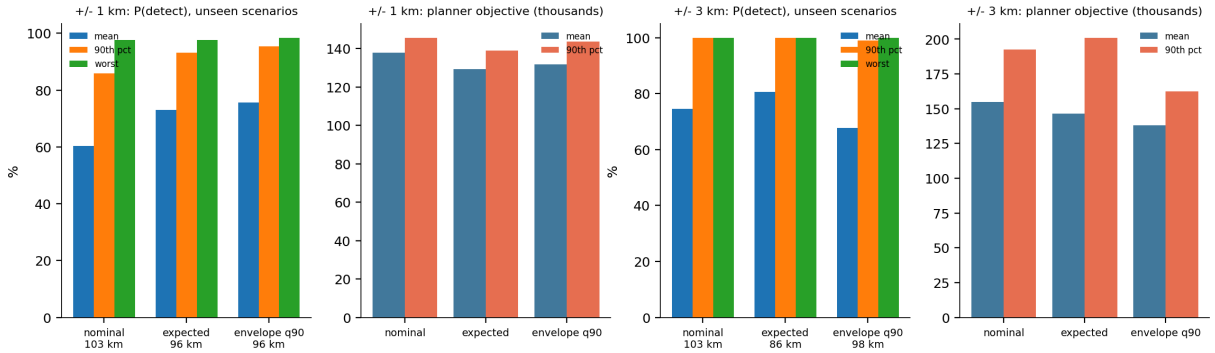

Figure 30. Uncertain radar positions: detection probability and planner objective on unseen scenarios.

Alternative corridors (E14). A planner should offer genuinely different options. The via-node method (choice routing) needs only two fields, one from the start and one to the goal, which give the best route through every cell; scanning cells at least 3 km from the best route found 2 alternatives (+1.0% and +2.0%) in 19.5 s. The penalty method (make cells near each route dearer and re-plan) found 3 alternatives at +2.0% to +6.3% with at most 4% overlap in 2.5 s, and is the better choice on this terrain: narrow valleys leave few near-optimal detours, which the via-node scan must search for.

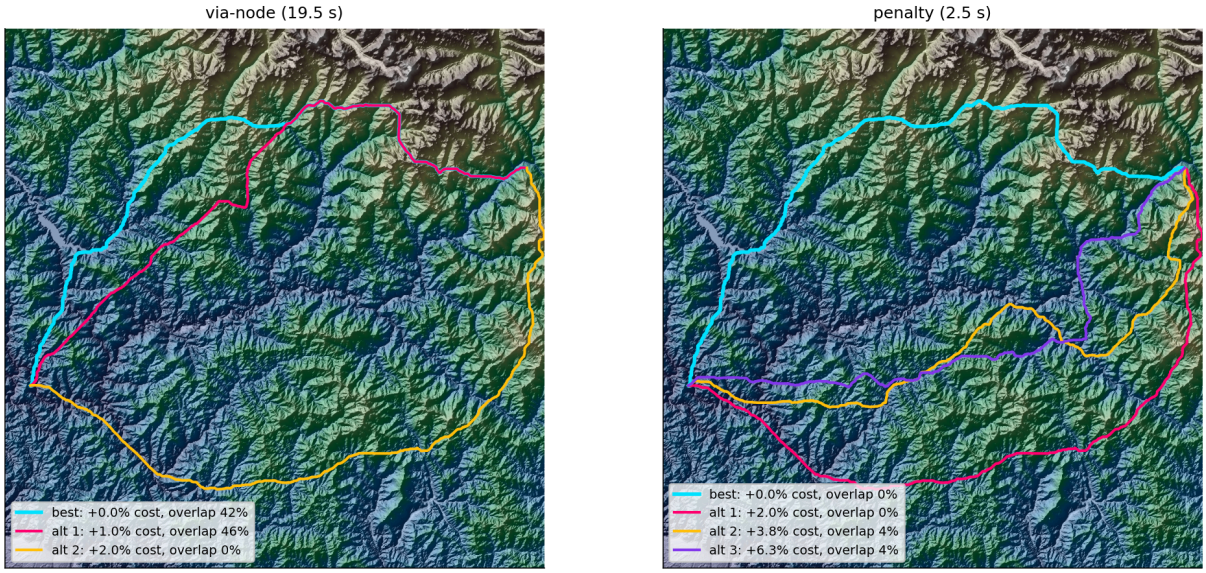

Figure 31. Three alternative corridors: via-node method (left) and penalty method (right).

Checking the detection model (E15). Simulating 200,000 flights per route (each radar detecting independently per segment) reproduces the formula within the 95% interval for every route (e.g. 87.7% simulated against 87.8%). The model itself matters: if the target's radar cross-section fluctuates from flight to flight (a Swerling-I-like model), the same route is detected with 78.2% instead of 87.8%. If the true radar reach is shorter than assumed (d50 = 10 km instead of 20), the nominal route is 8.0-13.7% worse than a re-optimised one and shares only 28% of it; if the reach is the same or longer it stays within 4.1%. Range calibration matters more than the peak detection probability.

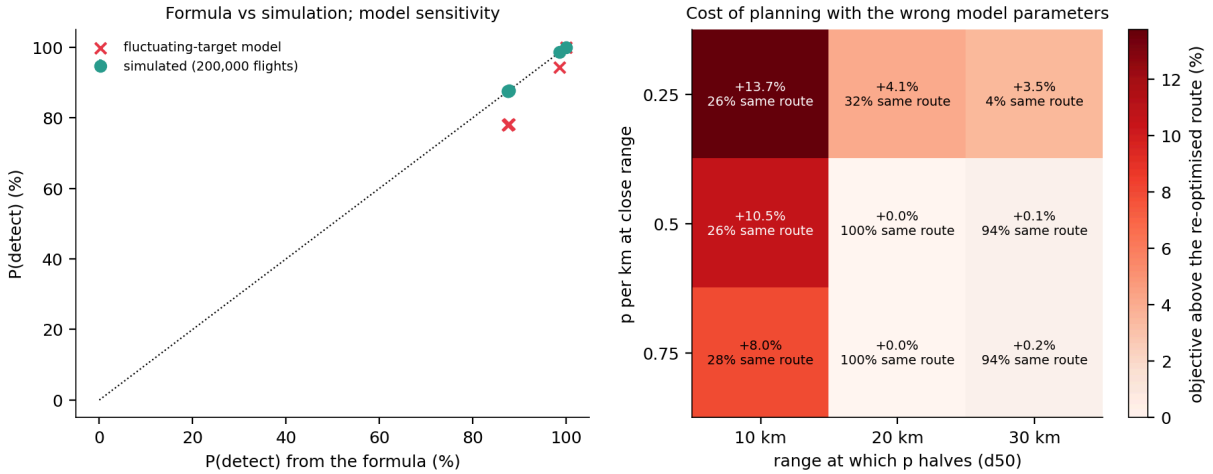

Figure 32. Monte Carlo against the formula, a fluctuating-target model, and the cost of wrong model parameters.

Summary: what to precompute, what to compute on the fly

- Precompute the cost-to-go field for the current goal (exact routes from any start by lookup) and a small bank of fields for other alpha values; keep the landmark set or the tile overlay for new goals.
- Compute on the fly only what an edit invalidates: one radar's viewshed window, then a parallel rebuild of the field (0.3-0.4 s on 12 cores).
- When the precompute does not apply, answer at once with an approximate route (bank, coarse-to-fine, overlay) and improve it to the exact one in the background.
- Pan and zoom never search: they draw the route already known.
- Report risk as a probability, and answer probability targets with a certified constrained search.

Appendix C. Reproducibility

All code, results, the QGIS project, this report and the slide deck are in the team repository: **https://github.com/Saloni-go/Optics_Spatial**. The DEMs are not stored there; the first command downloads and rebuilds them.

Table 28. Commands to rebuild every result.

Step	Command (Windows, QGIS 3.44 Python)
Data	run.bat scripts\prepare_data.py
Track 1	run.bat -m radar.validate | radar.run_radar | radar.benchmark | radar.analysis
Track 2	run.bat -m flight.run_flight | flight.benchmark | flight.spacing_study | flight.analysis
Track 3	run.bat -m corridor.run_corridor | corridor.benchmark | corridor.analysis
Track 3 (App. B)	run.bat -m corridor.interactive_bench all | corridor.extensions_bench all | corridor.interactive
QGIS	run_qgis.bat qgis\build_project.py ; run_qgis.bat qgis\render_bench.py
Report / slides	python report\build_report.py ; node presentation\build_pptx.js

Table 29. Repository layout.

Folder	Contents
common/	configuration, raster I/O, benchmark harness, plotting, shared radar sites
radar/	viewshed algorithms, validation, scenario, benchmarks, analysis
flight/	routes, moving-observer visibility, scenario, benchmarks, sampling study
corridor/	exposure surface, route solvers (Dijkstra, A*, landmark A*, SciPy, multi-resolution); compiled C core (c/), interactive planner and viewer, extensions (risk budget, tiles, altitude, placement, alternatives)
results/	GeoTIFF, GeoJSON, CSV, JSON and figures for every track
qgis/, report/, presentation/	QGIS project and render benchmark, this report, slide deck